

Additional Higher Grade Tasks Solution

Object-Oriented Design, IV1350

Mo Wang mowang@ug.kth.se

2026-05-07

Project Members:

[Mo Wang, mowang@ug.kth.se]

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

1 Introduction

This report concerns additional design pattern implementation as well as implicit unit tests in the *Repair Electric Bike* system. The purpose of Additional Higher Grade Task is to extend the current program implementation with additional design pattern, design pattern evaluation as well as writing indirect test cases in terms of testing View printouts. The implementation follows the principles and guidelines described in Chapter 9 (design patterns) of *A First Course in Object-Oriented Development* [1].

The additional higher-grade tasks consist of the following requirements:

- **Task 1: Template Method pattern.** Application of the **Template Method design pattern** to the observer mechanism implemented in earlier seminars. The implementation must contain the required **try--catch block** structure inside the template method with abstract methods defining the variations of different template patterns.
- **Task 2: Inheritance vs Composition.** Adaption of a class from the Java standard library using both **inheritance** and **composition**, while comparing the

difference in terms of code quality such as encapsulation and cohesion, according to the discussion in the coursebook Chapter 9.3. The solution includes a program as well as a printout providing the result behavior of both adaption implementations.

- **Task 3: Testing output.** Implementation of **unit tests for all methods that print to System.out** in the system, including the main workflow, the View class, and repair order printouts. The tests focus on informative output and are implemented using a testing framework such as JUnit.

2 Task 1: Template design pattern

The first task focuses on implementation of the Template method design pattern for the existing observers concerning RepairOrder in the program. The intent is to centralize and hide outer control-flow logic by creating an abstract superclass with the common code parts while the variant parts are delegated into abstract methods which are implemented in its subclasses for specific customized behaviors. This introduces centralized exception handling for observer updates.

2.1 Method

The template design pattern is introduced when duplicated code sections are detected where code structures are similar with varying inner logic. Ordinary method extraction, however, can only reduce smaller pieces of duplicated code. In this case, the template method pattern places the common workflow structure into an abstract superclass, while the varying logic is delegated into abstract methods implemented and overridden in concrete subclasses.

The pattern is applied on the `RepairOrderView` and `RepairOrderLogger`, that implements the interface `RepairOrderObserver`. The reason is that during their functional extension, the error catch blocks may be introduced during failure to display or logging. In such case, the catch block becomes the common duplicated control code.

The implementation work begins by identifying duplicated observer update logic in the existing view classes related to repair order notification. The common execution structure (the common control block), including exception handling and update sequencing, is extracted into a shared abstract superclass as the entry method, that delegates the inner varying logic as abstract methods. These abstract methods are implemented in subclasses, where the previously extracted observer-specific logic is reinserted depending on the use case.

The reason of selecting and applying the Template Method pattern for observer behaviors is to centralize, fix and reduce repetition of the exception handling control block of the algorithm, while allowing different observer behaviors to vary. The pattern is chosen instead of simple method extraction because the control-flow structures is repeated, which cannot be effectively factorized using ordinary internal methods.

The implementation is also continuously tested after each refactoring step to verify that observer updates still produces correct behavior.

2.2 Result

The implementation result shows that using the abstract class `AbstractRepairOrderObserver` with template control-flow, the observer interface can be implemented by defining the template method.

Listing 1: `View.proceedActions` catching exceptions and displaying error messages

```
1 public abstract class AbstractRepairOrderObserver implements
   RepairOrderObserver {
2     private RepairOrderDTO repairOrderDTO;
3
4     @Override
5     public final void updateRepairOrder(RepairOrderDTO repairOrderDTO) {
6         this.repairOrderDTO = repairOrderDTO;
7         handleRepairOrderUpdate();
8     }
9
10    private void handleRepairOrderUpdate() {
11        try {
12            doHandleRepairOrderUpdate();
13        } catch (Exception e) {
14            handleErrors(e);
15        }
16    }
17
18    protected RepairOrderDTO getRepairOrderDTO() {
19        return repairOrderDTO;
20    }
21
22    protected abstract void doHandleRepairOrderUpdate() throws Exception;
23    protected abstract void handleErrors(Exception e);
24 }
```

Whenever a repair order is modified, the method `updateRepairOrder` is called. The method `updateRepairOrder` is public and calls the private internal method, where the template common logic lives. The method `updateRepairOrder` stores the provided data transfer object and calls a private internal method `handleRepairOrderUpdate` where template common logic is defined. The common logic contains a try-catch block where catch block contains exception handling.

Two abstract methods are defined for concrete implementations. The abstract method `doHandleRepairOrderUpdate` handles the error-free observer-specific update logic while `handleErrors` handles any exception that occurs during execution.

Using the template method, two new concrete observers are adapted. The class `RepairOrderLogger` implements the template hooks (abstract methods) to log repair order updates to a file, while `RepairOrderViewTemplate` implements the hooks to print user-facing notifications to standard output.

A sample run of the application shows that repair order updates are automatically propagated to both observers, with logging and user notification handled correctly. The

messages of repair order updates are not modified, but the implementation is different.

Listing 2: Printout of repair order application

```
--- REPAIR ORDER MODIFICATION NOTIFICATION (BEGIN) ---  
THE REPAIR ORDER HAS BEEN MODIFIED      : 8b87d176-8fdc-4f2f-afd1-10123ca8bebc  
PLEASE USE TELE NUMBER TO CHECK DETAILS: +46707654321 (Astrid Johansson)  
--- REPAIR ORDER MODIFICATION NOTIFICATION -(END)- ---
```

The complete source code for this seminar is available in the public Git repository at: <https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/> [2].

2.3 Discussion

The implemented solution applies the Template Method design pattern for several reasons. The abstract base class defines the common algorithm structure for handling observer notifications, including exception handling, while delegating the variable behavior to subclasses.

The implementation preserves the required pseudocode structure. The template method containing the `try--catch` block for exception handling is implemented, as well as the abstract methods that the template method calls. The entry method is given by the public final method `updateRepairOrder`. The template method is represented by the private method `handleRepairOrderUpdate` which contains the common try-catch block and is discouraged from being externally called. The protected method `doHandleRepairOrderUpdate` provides the algorithm for handling updated repair orders during error-free conditions, while the method `handleErrors` concerns error scenarios. Both abstract methods `doHandleRepairOrderUpdate` and `handleErrors` correspond to the required variation points, ensuring that subclasses provide their specific implementations.

This shows that the algorithm control-flow skeleton is fixed in the abstract class and cannot therefore be modified by its subclasses, while only the individual steps are customizable. This separation between invariant control flow and variant behavior is the defining characteristic of the Template Method design pattern.

In terms of encapsulation and method safety, the template method `handleRepairOrderUpdate` is set as private ensuring that it cannot be called arbitrarily outside the class, while the public entry method `updateRepairOrder` that delegates the template method is set as final to avoid having the template behavior being altered, in order to ensure encapsulation and safety.

Using the Template Method pattern provides several benefits in this application. It eliminates duplicated control flow logic that would otherwise appear in each observer implementation. It also enforces consistent error handling mechanism across all observers, improving robustness and maintainability.

Additionally, the design improves extensibility. New observers can be added by subclassing the abstract base class and implementing the two abstract methods, without

modifying existing code. This aligns well with the Open–Closed Principle and supports future extensions of the system.

Overall, the Template Method pattern is well suited for structuring the observer update logic in this application and results in a clearer, more maintainable design.

3 Task 2: Adaption using inheritance compared to composition

The second task compares adaption implementation using inheritance with composition of a Java library class. Two separate adapting classes are implemented using the same Java library class, which allows the comparison of flexibility, encapsulation, coupling, and maintainability between the two approaches. The goal is to demonstrate how inheritance can introduce hidden coupling to internal implementation details of a class, while composition avoids this problem.

3.1 Method

To address the task, a class from the Java standard library, `java.io.RandomAccessFile`, was adapted in two different ways, using inheritance and composition respectively. Showing that inheritance introduces hidden coupling due to dependency on the internal implementation details of the superclass rather than only its public interface, a protocol reading functionality is implemented for the adapter. The method `readProtocol` is implemented in both versions for reading and interpreting the bytes of a protocol, while tracking the number of bytes read during protocol parsing.

First of all, an adapter based on inheritance was implemented by creating a subclass of `RandomAccessFile`. This adapter overrides selected methods in order to interpret a custom binary message protocol and to track how many bytes are read during protocol parsing.

Second, an adapter based on composition was implemented. Instead of extending `RandomAccessFile`, this adapter contains a `RandomAccessFile` instance as a private field and delegates all file operations to that instance. The same protocol parsing functionality and byte-counting logic is implemented, but without overriding any superclass methods.

A `main` method was written to create a binary test file containing two protocol messages and to read the file sequentially using both adapters. The program prints the interpreted protocol messages and the number of bytes counted by each adapter, thereby illustrating the behavioral differences between the two adaptation approaches.

This implementation shows a controlled comparison between the two design approaches in terms of encapsulation, coupling, and maintainability, since both adapters implement identical functionality, and the only difference lies in the use of inheritance versus composition.

3.2 Result

The inheritance-based adapter and the composition-based adapter both interpret the same protocol format and count the number of bytes read during parsing. However,

they produce clearly different behavior in terms of byte counting.

In the inheritance-based adapter, the protocol messages are interpreted correctly, but the internal byte counter remains zero for both reads. This occurs even though data is clearly being read from the file. The reason is that only the method `read` is overridden by the subclass for counting number of bytes, while the subclass depends on the internal implementation of the superclass rather than its public contract. This can be shown in the code implementation that the method `readFully` does not call `read` as expected, but bypasses it. As a result, the byte counter variable is not incremented correctly, although data is being read from the file.

Listing 3: Implementation of `InheritedProtocolInterpreterAdapter`

```
1 public class InheritedProtocolInterpreterAdapter extends RandomAccessFile {
2     private int bytesRead = 0;
3     /**
4      * Overrides {@code read(byte[])} to count bytes read.
5      * <p>
6      * The subclass expects this to be called by {@code readFully()}, but the
7      * superclass
8      * bypasses it -- demonstrating that inheritance exposes you to
9      * undocumented internal behavior.
10     */
11     @Override
12     public int read(byte[] b) throws IOException {
13         bytesRead += b.length;
14         return super.read(b);
15     }
16     /**
17      * Reads a {@link MessageProtocol} from the binary file.
18      * <p>
19      * The binary format is: 1 byte title length, title bytes, 2 byte
20      * description length, description bytes.
21      * Uses {@code readFully()} which does NOT call the overridden {@code read(
22      * byte[])},
23      * so bytesRead remains zero -- an incorrect result caused by inheritance.
24      * <p>
25      * @return the parsed message protocol, or a default protocol if an I/O
26      * error occurs
27     */
28     public MessageProtocol readProtocol() {
29         try {
30             // ...
31             readFully(titleBytes);
32             // ...
33         } catch (IOException e) {
```

```
33         return new MessageProtocol("No title", "No description", bytesRead)
34     ;
35     }
36 }
```

When using the composition-based adapter, the protocol messages are interpreted correctly and the byte counter reflects the actual number of bytes read. The counter increases consistently as successive protocol messages are parsed.

Listing 4: Implementation of CompositeProtocolInterpreterAdapter

```
1 public class CompositeProtocolInterpreterAdapter implements AutoCloseable {
2     private final RandomAccessFile randomAccessFile;
3     private int bytesRead = 0;
4
5     /**
6      * Reads a {@link MessageProtocol} from the binary file.
7      * <p>
8      * The binary format is: 1 byte title length, title bytes, 2 byte
9      * description length, description bytes.
10     * </p>
11     * @return the parsed message protocol, or a default protocol if an I/O
12     * error occurs
13     */
14     public MessageProtocol readProtocol() {
15         try {
16             // ...
17             readBytes(titleBytes);
18             // ...
19         } catch (IOException e) {
20             return new MessageProtocol("No title", "No description", bytesRead)
21         ;
22         }
23     }
24
25     private void readBytes(byte[] b) throws IOException {
26         randomAccessFile.readFully(b);
27         bytesRead += b.length;
28     }
29 }
```

A sample run printout illustrating this behavior is included below. The run printout shows that the byte count shows 0 for inheritance adapter, while the printout number is correct for composition adapter. The printout and results show the pathological design limitation using inheritance during class adaption.

Listing 5: Sample output for comparing inheritance and composition

```
--- Inheritance vs Composition Demo ---
(Each adapter overrides/wraps read() to track bytesRead)

[Inheritance adapter (extends RandomAccessFile)]:
  Overrides read(byte[]) to count bytes, but readFully()
  does NOT call read(byte[]) -- the override is SILENTLY BYPASSED.

  First read: MessageProtocol(title = BikeAlert, description = Chain
    replacement needed, bytesRead = 0)
  Second read: MessageProtocol(title = BatteryAlert, description = Battery
    replacement needed, bytesRead = 0)

[Composition adapter (wraps RandomAccessFile)]:
  Counts bytes in its own readBytes() method -- no reliance
  on superclass internals, so counting is always correct.

  First read: MessageProtocol(title = BikeAlert, description = Chain
    replacement needed, bytesRead = 33)
  Second read: MessageProtocol(title = BatteryAlert, description = Battery
    replacement needed, bytesRead = 71)

...
```

The full source code for both adapters and the demonstration program is available in the public Git repository: <https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/InheritanceComposition> [3].

3.3 Discussion

The observed behavior clearly illustrates the argument presented in Chapter 9.3 of the course literature. Although both adapters use the same underlying library class, the inheritance-based approach introduces a hidden dependency on internal implementation details of `RandomAccessFile`.

In the inheritance-based adapter, the method `read(byte[])` is overridden to count number of bytes read. However, the method `readFully(byte[])` that is used for protocol parsing does not call `read(byte[])` internally. Instead, it relies on a different internal implementation path, which results into silent bypass of the method `read(byte[])` and the byte counter is never updated. This behavior is not caused by an error in the subclass itself, but by its reliance on undocumented internal behavior of the superclass.

This approach shows how inheritance breaks encapsulation. Since the subclass implicitly depends on how the superclass implements its internal logic, the behavior of the subclass depends heavily on undocumented internal logic of superclass, outside of the control from the subclass. If a future version of the Java library were to change the internal implementation of `readFully(byte[])`, the subclass behavior would change again, either starting to count bytes correctly or even double-counting, without any modification to the subclass. In this case, the encapsulation is broken, since the subclass relies on the implementation of the superclass.

Furthermore, inheritance forces the subclass to accept the entire public interface of the superclass, including methods that may not be relevant to the adaptation. This reduces control over the exposed interface and makes the subclass more sensitive to changes in the superclass, which further increases coupling.

In contrast, the composition-based adapter avoids this problem. All interactions with `RandomAccessFile` occur through its public interface, and the adapter explicitly controls when and how the byte counter is updated. The behavior of the adapter does not depend on the internal call structure of the wrapped object, and future changes to the library implementation are therefore much less likely to cause unintended side effects.

In conclusion, composition is the preferred adaptation approach. Despite the apparent simplicity of inheritance, it causes tight coupling to superclass implementation and violates encapsulation, making the adaptation implementation more fragile and harder to maintain. Composition preserves encapsulation and results in a more robust and predictable design, which is consistent with the conclusion drawn in Chapter 9.3 of the course literature.

4 Task 3: Testing Output

The third task focuses on automated unit testing of program output written to `System.out` in the *Repair Electric Bike* application. Unit tests are written for all methods that print information to `System.out`, including the main program flow and view-related functionality. The purpose is to verify that the program produces correct output during execution and that the user interface behaves as expected, during both normal execution and error-handling scenarios.

4.1 Method

Unit tests are implemented using the JUnit framework to verify all methods that produce informative output to `System.out`. The general approach is to treat output-producing classes as black boxes and validate observable behavior rather than internal implementation details.

During each test, `System.out` is temporarily redirected to a `ByteArrayOutputStream` before invoking the method under test and the original output stream is restored afterwards. This allows printed output to be captured, checked, and therefore asserted without modifying the SUT code.

Only functions that produce relevant information are tested, complying with the task requirements. Output that exists only for improving readability in view printouts, such as separators and decorative texts, are deliberately excluded. Additionally, both successful workflows and error scenarios are tested to ensure correct behavior in all situations.

4.2 Result

The implemented unit test classes include `MainTest` for testing the `Main` class, `ViewTest` for testing the `View`, as well as `RepairOrderViewTemplateTest`, `RepairOrderLoggerTest`,

and `PrinterTest`. Together, these tests cover all methods that produce informative output to `System.out` in the system.

- `MainTest` verifies the output produced by the application entry point during execution of the full repair workflow. The test ensures that the expected sequence of operations, such as customer lookup, repair order creation, and notification messages, are printed correctly.
- `ViewTest` verifies all informative output produced by the `View` class. The tests cover both normal workflows and error-handling scenarios, such as invalid input, missing customer data, and invalid repair order identifiers. This ensures that the `View` behaves correctly under different conditions.
- `RepairOrderViewTemplateTest` verifies observer-based notification output triggered by repair order updates. The tests assert that important information such as repair order ID, customer name, and telephone number are printed correctly. In addition, error handling is tested by passing a null value, verifying that an appropriate error message is produced.
- `RepairOrderLoggerTest` verifies output generated when logging fails. The test ensures that informative error messages are printed to `System.out` when exceptions occur during logging.
- `PrinterTest` verifies that repair order printouts contain all relevant information, such as customer details and repair order identifiers, ensuring correctness of the printed receipt.

All tests follow the same testing strategy, where `System.out` is redirected to a `ByteArrayOutputStream`, allowing printed output to be captured and verified using assertions. The tests are self-evaluating and produce no output when passing.

Each test case checks the correctness of output by searching and identifying message keywords. If the test fails to identify the required message parts, the printout message is guaranteed to be malformed, whereas a failed error message is reported. Otherwise, the printout message is trusted and treated as valid. Using this approach, incorrectly produced messages, such as calling wrong function after refactoring, can be detected and fixed at early stage.

As an example, the `RepairOrderViewTemplateTest` verifies multiple aspects of the notification output by checking individual pieces of information. A total of six test cases are executed, covering both normal and error scenarios.

Listing 6: Example JUnit test execution result

```
[INFO] Running se.ebikerepair.view.RepairOrderViewTemplateTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
```

The successful test execution shows that the observer notification output behaves correctly and consistently across different test cases.

The complete test implementations are available in the public Git repository: [https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/\[2\]](https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/[2]).

4.3 Discussion

The implemented unit tests are evaluated according to the assessment criteria for Seminar 3, particularly in terms of testing output-producing methods.

Readability and naming conventions The test code is easy to understand and follows consistent naming conventions. Each test class clearly reflects the class under test, where the suffix `-Test` is added to the name of the tested class, such as `ViewTest` and `PrinterTest`. Individual test methods describe the tested scenario, which improves readability and maintainability.

Code duplication and structure In terms of code redundancy, there is no significant duplication in test code. Common test setup, such as redirection and restoration of `System.out`, is handled using `@BeforeEach` and `@AfterEach`. These setup annotations keep test methods focused and prevents unnecessary repetition.

Cohesion, coupling, and encapsulation Each test class is responsible for testing exactly one production class, which results in high cohesion. Tests interact only with public interfaces and observable output, preserving encapsulation and keeping coupling low. Internal implementation details are not accessed directly, which allows production code to evolve without breaking tests.

Testing framework and self-evaluating tests JUnit is used consistently as the testing framework. All tests are self-evaluating: they rely exclusively on assertions and do not print any output when passing. If a test fails, the assertion messages provide sufficient information to diagnose the error. This ensures that the tests automatically detect failures while producing no output when all tests pass, which aligns with the definition of self-evaluating tests in the assessment criteria.

Project structure and organization All tests are placed in the same packages as the system under test, but in a separate test source directory. This follows recommended project structure and simplifies navigation between production code and tests. There is one test class per tested class, which improves clarity.

Coverage of execution paths and parameters The tests cover both normal execution paths and relevant error cases. Meaningful and representative parameter values are used, including valid input, invalid input, and missing data. By testing both successful and failing scenarios, the tests provide confidence that informative output behaves correctly under realistic conditions.

Overall assessment Overall, the unit tests are meaningful, systematic, and well-motivated. They fulfill the requirements of Task 3 and align well with the assessment criteria from Seminar 3. The testing approach ensures that all informative output written to `System.out` is verified in a robust and maintainable manner.

References

- [1] Leif Lindbäck, *A First Course in Object-Oriented Development — A Hands-On Approach*, Course material for IV1350, KTH Royal Institute of Technology, January 14, 2026.
- [2] Mo Wang, “ElectricBikeRepair — Source code for Seminar 5 for task 1 and 3,” 2026. [Online]. Available: <https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/ElectricBikeRepair/>
- [3] Mo Wang, “InheritanceComposition — Source code for Seminar 5 for task 2,” 2026. [Online]. Available: <https://github.com/mo-mosverkstad/java-learning/tree/main/maven-projects/InheritanceComposition>